



universidad autónoma de baja california

ingeniera en computación

traductores

guillermo licea sandoval

proyecto final: pseudothink run

garcia chavez erik

28 de noviembre del 2025

PseudoThinkRun es un intérprete de pseudocódigo diseñado para facilitar el acercamiento a la programación, Permite escribir programas utilizando sintaxis en español, soportando estructuras de control, funciones y estructuras de datos personalizadas.

El sistema funciona mediante una arquitectura de cuatro fases:

1. **Análisis Léxico (Lexer):** Convierte el código fuente en tokens.
2. **Análisis Sintáctico (Parser):** Valida la gramática y genera una Representación Intermedia (IR).
3. **Representación Intermedia (Tuplas):** Convierte el código en una lista lineal de instrucciones ejecutables.
4. **Interpretación:** Ejecuta las tuplas secuencialmente.

Guía de Inicio Rápido (Compilación y Ejecución)

Requisitos

- Java JDK 8 o superior.

Compilación

Para compilar el proyecto desde la raíz del código fuente (src/), utilice el siguiente comando:

```
# Crear directorio de salida  
mkdir out
```

```
# Compilar todas las clases  
javac -d out -sourcepath src src/com/pseudothinkrun/Main.java
```

Ejecución

El intérprete espera un archivo con extensión .psdo.

```
java -cp out com.pseudothinkrun.Main mi_programa.psd
```

Manual de Usuario (Lenguaje .psdo)

Esta sección describe la sintaxis del lenguaje que interpreta el sistema.

Tipos de Datos

El sistema soporta los siguientes tipos primitivos:

- ENTERO: Números sin decimales (ej. 10, -5).
- REAL: Números con decimales (ej. 3.14, -0.01).
- CADENA: Texto entre comillas dobles (ej. "Hola Mundo").
- LOGICO: Valores de verdad (VERDADERO, FALSO).

Declaración y Asignación

Las variables deben declararse antes de usarse con DEFINIR ... COMO

```
DEFINIR edad COMO ENTERO
edad = 25
```

```
DEFINIR nombre COMO CADENA
nombre = "Juan"
```

3.3 Entrada y Salida

- **ESCRIBIR:** Imprime valores en consola. Soporta múltiples argumentos separados por coma.
- **LEER:** Lee un valor de la entrada estándar y lo guarda en una variable.

```
ESCRIBIR "Ingrese su nombre:"
LEER nombre
ESCRIBIR "Hola ", nombre
```

Estructuras de Control

Condicional SI

```
SI edad >= 18 ENTONCES
    ESCRIBIR "Es mayor de edad"
SINO
    ESCRIBIR "Es menor de edad"
FINSI
```

Ciclo MIENTRAS

```
DEFINIR contador COMO ENTERO
contador = 0
MIENTRAS contador < 5 HACER
    ESCRIBIR "Contador: ", contador
    contador = contador + 1
FINMIENTRAS
```

Ciclo REPITE

```
REPITE 3 VECES
    ESCRIBIR "Esto se imprime 3 veces"
FINREPITE
```

Funciones

Las funciones pueden recibir parámetros y retornar valores. Tienen su propio ámbito (scope).

```
FUNCION sumar(a COMO ENTERO, b COMO ENTERO) ENTERO
    RETORNAR a + b
FINFUNCION
```

```
DEFINIR resultado COMO ENTERO
resultado = sumar(10, 5)
```

Estructuras (Tipos Personalizados)

Permite definir registros (structs) con múltiples campos.

```
TIPO Persona
    DEFINIR nombre COMO CADENA
    DEFINIR edad COMO ENTERO
FINTIPO
```

```
DEFINIR p1 COMO Persona
p1.nombre = "Ana"
p1.edad = 30
```

Manual Técnico (Arquitectura del Sistema)

Esta sección detalla las clases principales y el flujo de datos interno, ideal para mantenimiento o expansión del sistema.

Paquete: **com.pseudothinkrun.lexer**

Encargado de transformar el texto plano en una secuencia de objetos significativos.

- **Lexer.java:** Lee el archivo caracter por caracter.
 - *Función Clave:* `nextToken()`: Identifica si la secuencia actual es un número, una cadena, un operador o una palabra reservada. Maneja comentarios y espacios en blanco.
- **Token.java:** Objeto que representa una unidad léxica (ej. un identificador, un SI, un número). Guarda la línea y columna para reportar errores.
- **TokenType.java:** Enum que define todos los tipos de tokens posibles (SI, MIENTRAS, IDENTIFICADOR, etc.).

Paquete: **com.pseudothinkrun.parser**

Encargado de validar la gramática y construir la lógica del programa.

- **PseudoParser.java:** El corazón del análisis.
 - Utiliza un algoritmo de **Descenso Recursivo**.
 - No genera un Árbol de Sintaxis Abstracta (AST) complejo, sino que traduce directamente a una lista plana de **Tuplas** (Instrucciones Intermedias).
 - *Funciones Clave:*
 - `parsePrograma()`: Punto de entrada.
 - `parseSi()`, `parseMientras()`: Gestionan estructuras anidadas y bloques de código.
 - `parseFuncion()`: Registra la función en la tabla de símbolos y guarda sus tuplas internas para ejecución diferida.

Paquete: **com.pseudothinkrun.intermediate (Las Tuplas)**

El sistema utiliza un enfoque de **Código Intermedio** basado en clases llamadas "Tuplas". Cada Tupla es una instrucción ejecutable.

- **Tupla.java:** Clase abstracta base. Define el contrato `ejecutar(SymbolTable)`. Devuelve un entero que indica qué hacer a continuación (seguir, saltar, error).
- **Tipos de Tuplas Importantes:**
 - **AsignacionTupla:** Evalúa expresiones (matemáticas o llamadas a función) y actualiza la `SymbolTable`. Maneja asignaciones a estructuras (`rect.base`).
 - **SiTupla:** Contiene listas de otras tuplas (entonces, sino) y decide cuál lista ejecutar basándose en una `ComparacionTupla`.
 - **LlamadaFuncionTupla:**
 1. Resuelve la función desde la tabla de símbolos.
 2. Crea un **nuevo Scope** (ámbito) exclusivo para esa ejecución.

3. Mapea los argumentos a los parámetros.
4. Ejecuta las tuplas internas de la función.

Paquete: **com.pseudothinkrun.symbols**

Gestiona la memoria y los ámbitos (scopes) de las variables.

- **SymbolTable.java:**
 - Mantiene una pila (Stack) de ámbitos (Scopes).
 - Permite que las variables dentro de una función no interfieran con las variables globales.
- **VariableSymbol.java:** Almacena el nombre, tipo y el **valor** (Object) de una variable en tiempo de ejecución.
- **FunctionSymbol.java:** Almacena la definición de la función, sus parámetros y la **lista de Tuplas** que componen su cuerpo. Actúa también como un Scope.
- **StructSymbol.java:** Permite la definición de tipos complejos. Actúa como un Scope para sus miembros internos.

Paquete: **com.pseudothinkrun.interpreter**

El motor de ejecución.

- **Interpreter.java:**
 - Recibe la lista de tuplas generada por el Parser.
 - Posee un bucle principal (while) que actúa como *Program Counter* (PC).
 - Delega la ejecución lógica a cada tupla: `tuplaActual.ejecutar(symbolTable)`.

4.6 Flujo de Ejecución Detallado

1. **Main** lee el archivo .psdo.
2. **Lexer** genera Tokens.
3. **Parser** lee Tokens y:
 - Si encuentra DEFINIR, crea símbolos en la SymbolTable.
 - Si encuentra FUNCION, guarda la definición pero no la ejecuta.
 - Si encuentra instrucciones (a = 1, SI...), crea objetos Tupla y los añade a una lista.
4. **Interpreter** toma la lista de Tuplas:
 - Recorre la lista una por una llamando a `ejecutar()`.
 - Si es una AsignacionTupla, calcula el valor y actualiza la variable en memoria.
 - Si es una EscribirTupla, muestra el resultado en consola.
 - Si es una LlamadaFuncionTupla, el intérprete pausa el flujo actual, crea un entorno aislado, ejecuta la función y recupera el valor de retorno.